



Tecnológico de Monterrey

Ejercicio 4: Normalización de la base de datos del reto apoyándose en recursos de IA generativa

Yael Ordaz – A01786776

J. Manuel Montero - A01660761

Santiago Hernández – A01787550

|

Equipo 7

Abril 18, 2026

Construcción de Software y Toma de Decisiones (Gpo 401)



Después de varias semanas de trabajo, fuimos capaces de mejorar nuestro esquema de la base de datos de nuestro juego (Florida Frenzy). Esto gracias a todo lo que estudiamos en clase, nuestras diferentes experiencias de trabajo en base de datos, y las recomendaciones y observaciones hechas por el profesor Esteban. De inicio, logramos acercarnos a tener un esquema en 3FN. Parte de los últimos comentarios del profesor hacían hincapié en lograr una normalización más adecuada para la base de datos, considerando adoptar un mayor nivel de granularidad. Esto implicaba distribuir ciertos valores en más tablas y agregar algunos detalles menores en la estructura general del modelo.

Sin embargo, nuestro juego está integrado con Prisma para el backend, por lo que tuvimos que trasladar ese esquema inicial de PostgreSQL a un modelo compatible con esta herramienta, sin perder de vista el objetivo de satisfacer la tercera forma normal (3FN). En este proceso, fue necesario equilibrar dos necesidades: por un lado, mantener una base de datos correctamente normalizada desde el punto de vista académico; y por otro, conservar una estructura práctica, clara y funcional para la implementación real del proyecto.

Como resultado, se tomó la decisión de conservar separadas las entidades principales del sistema, modelar explícitamente las relaciones de muchos a muchos mediante tablas intermedias y evitar redundancias innecesarias en la mayor parte del diseño. Al mismo tiempo, algunos elementos se mantuvieron con una estructura ligeramente más flexible para no complicar de forma excesiva la lógica del backend ni el desarrollo de las funcionalidades del juego.

De esta forma, el esquema final no solo responde a los principios de normalización aprendidos en clase, sino también a las necesidades técnicas de integración con Prisma, permitiendo que la base de datos fuera consistente, escalable y viable dentro del flujo real de desarrollo.

Y luego se nos asignó esta tarea, la cual consiste en presentar un esquema mejorado, con sugerencias de un modelo generativo, o IA. Utilizamos ChatGPT (5.4). De inicio, so veredicto general fue el siguiente:

“El esquema se encuentra mayormente normalizado hasta 3FN; sin embargo, se identificaron algunos atributos y relaciones susceptibles de refinarse para reducir redundancia semántica, dependencias condicionales y posibles anomalías de actualización.”

Dejándonos con las siguientes mejoras:

Mejora 1: revisar PlayerCard.isUnlocked

Yo sí propondría eliminarlo, o justificarlo muy bien.

Motivo

Porque numCardsOwned ya comunica posesión/disponibilidad.

Beneficio



- menos redundancia
 - menos inconsistencias
 - mejor defensa de 3FN
-

Mejora 2: revisar PlayerCharacter.isUnlocked

Aquí depende de su lógica.

Si la tabla solo guarda personajes del jugador:

elimínenlo.

Si la tabla guarda el estado de todos los personajes por jugador:

pueden dejarlo.

Mejora 3: Battle debería referenciar RunZone

Esta es una mejora bastante buena.

En vez de:

- runId
- zoneGameId

Usar:

- runZoneId

Beneficios

- menos redundancia
 - más integridad referencial
 - mejor coherencia con la progresión del juego
-

Mejora 4: dividir Reward por subtipo o usar tablas hijas

Esta es la mejora estructural más fuerte.

Motivo

Evitar columnas opcionales según tipo de recompensa.

Beneficio



- más claridad
 - menos nulos
 - reglas más limpias
 - mejor normalización semántica
-

Mejora 5: agregar restricciones únicas

Esto no es exactamente normalización, pero sí mejora mucho el diseño.

Les sugeriría al menos:

- Player.email único
- Player.username único
- Player.firebaseUid único
- PlayerCharacter(playerId, characterGameId) único
- PlayerCard(playerId, cardGameId) único
- DeckCard(deckId, cardGameId) único
- RunZone(runId, order) único
- quizá BattleReward(battleId, rewardId) único

Y si aplica la lógica del juego:

- solo un Deck.isActive = true por jugador o por jugador+personaje

Y para normalizar, dejó esto:

Modificación A: simplificar PlayerCard

Actual

PlayerCard(id, playerId, cardGameId, numCardsOwned, isUnlocked)

Propuesta

PlayerCard(id, playerId, cardGameId, numCardsOwned)

Justificación

isUnlocked puede inferirse de numCardsOwned.

Ganancia



Se evita contradicción entre cantidad y estado.

Modificación B: revisar PlayerCharacter

Actual

PlayerCharacter(id, playerId, characterGameId, isUnlocked, usedTimes)

Propuesta posible 1

Dejarla igual, pero justificar:

la tabla guarda el estado de disponibilidad de cada personaje para cada jugador, incluso cuando aún no está desbloqueado.

Propuesta posible 2

Si no necesitan eso:

PlayerCharacter(id, playerId, characterGameId, usedTimes)

Justificación

La existencia del registro ya representa que el personaje está desbloqueado.

Modificación C: refactor de Battle

Actual

Battle(id, ..., runId, zoneGameId, enemyId)

Propuesta

Battle(id, ..., runZoneId, enemyId)

Justificación

Una batalla ocurre en una zona concreta de una run concreta. Esa información ya está contenida en RunZone.

Ganancia

Reduce redundancia lógica y evita inconsistencias.

Modificación D: refactor de Reward

Actual

Reward(id, rewardName, rewardType, rewardDesc, rewardValue, cardGameId, characterGameId)



Propuesta

Tabla base:

Reward(id, rewardName, rewardType, rewardDesc, rewardValue)

Tablas especializadas:

- RewardCard(rewardId, cardGameId)
- RewardCharacter(rewardId, characterGameId)

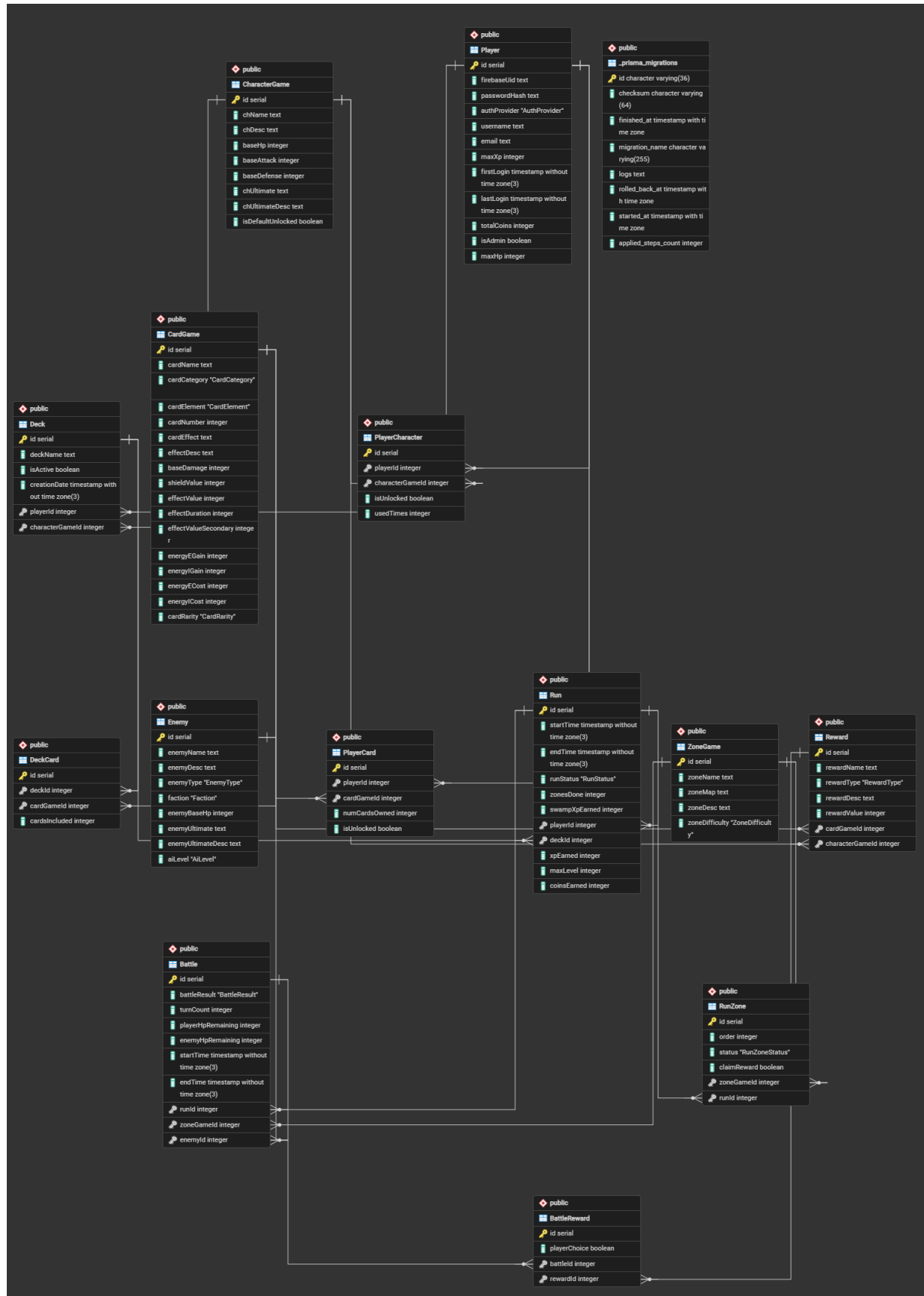
Justificación

Cada tipo de recompensa se relaciona con su entidad correspondiente sin depender de columnas opcionales.

Ganancia

Mejor claridad y menor dependencia condicional.

Nuestro esquema al momento de consultar con ChatGPT:





Todos estos cambios y observaciones están siendo considerados por el equipo, puesto que actualmente ya contamos con una base de datos funcional, así que debemos de evaluar primeramente si los cambios sugeridos, afectan severamente la funcionalidad de esta misma.

Con todo esto, nos damos cuenta de que, en un proyecto como este, la normalización resulta especialmente importante. Debido a que el videojuego maneja distintas entidades relacionadas entre sí, como jugadores, personajes, cartas, enemigos, recompensas, zonas, mazos y batallas. Si esta información se almacena sin una estructura adecuada, pueden aparecer problemas como duplicación de datos, errores al actualizar registros y dificultad para representar correctamente las relaciones del juego.

Si la información de las cartas de un jugador, sus mazos y las recompensas obtenidas se almacenara en una sola tabla, sería muy fácil provocar anomalías. Una misma carta podría repetirse muchas veces, una actualización tendría que realizarse en varios lugares y una eliminación podría borrar información valiosa de manera accidental. Gracias a la normalización, estos datos se separan en entidades específicas y en tablas puente cuando existe una relación de muchos a muchos, como sucede entre mazos y cartas, o entre jugadores y cartas.

Además, una base de datos normalizada facilita el desarrollo del backend, la definición de reglas de negocio y la construcción de consultas más confiables. Aunque en algunos casos puede ser útil conservar ciertos datos resumidos por motivos de rendimiento, partir de un diseño normalizado permite decidir conscientemente cuándo conviene desnormalizar y cuándo no.

Las herramientas de lenguaje generativo funcionaron como apoyo para revisar, cuestionar y mejorar el modelo entidad-relación, pero no sustituyeron el trabajo de análisis ni las decisiones de diseño realizadas por los desarrolladores.

Su principal contribución consistió en servir como un mecanismo de retroalimentación técnica. A partir de la descripción textual de las entidades del sistema y de las relaciones planteadas por el equipo, la herramienta ayudó a identificar posibles redundancias, dependencias transitivas, atributos derivables y oportunidades de refactorización. En particular, permitió señalar casos donde ciertos campos podían ser redundantes, donde algunas relaciones podían representarse de forma más consistente y donde convenía separar entidades según su función dentro del juego.

Sin embargo, las sugerencias generadas no se adoptaron automáticamente. Cada recomendación tuvo que evaluarse en el contexto real del proyecto, tomando en cuenta la lógica del videojuego, la implementación prevista en el backend, la experiencia de usuario y las restricciones del equipo. Es decir, la herramienta no diseñó la base de datos por sí sola, sino que apoyó el proceso de revisión crítica.